

mwb-ai-claw 技术方案

本文档为 mwb-ai-claw 项目的完整技术方案，涵盖项目背景、目标、概要设计、整体架构设计（整洁架构 + DDD + 六边形架构融合）、数据模型设计（UML 类图）以及各核心领域的详细流程设计（状态图、数据交互图、文字说明）。所有图形均采用 PlantUML 描述，可在支持 PlantUML 的编辑器（VS Code / IDEA / Typora 等）中直接渲染。

1. 项目背景

随着大语言模型（LLM）的成熟，AI Agent（智能体）从「聊天问答」进化为「能真正动手干活」的自主助手已成为趋势。然而，市面上多数 Agent 框架存在以下痛点：

- 过度依赖云端**：用户数据、会话历史、长期记忆全部上传到远端服务器，隐私无法保障。
- 工具扩展困难**：每接入一个新工具都要侵入式修改核心代码，缺乏标准协议。
- 上下文窗口受限**：长对话很快超出 LLM 上下文窗口，导致「失忆」。
- 单 Agent 独木难支**：复杂任务需要多个专家 Agent 协作，但编排能力弱或耦合死。
- 技术栈偏 Python**：Java 生态缺乏一个结构清晰、可扩展、本地优先的 Agent 框架。

mwb-ai-claw 正是在此背景下诞生的 **Java 版智能体 Agent 框架**，灵感来自 OpenClaw，旨在提供一个本地优先、多 Agent 协作、开箱即用的个人 AI 助手框架，并以 DDD + 整洁架构 + 六边形架构为骨架，确保高度可扩展性。

2. 项目目标

维度	目标	关键能力
本地优先	数据本地化，隐私可控	会话/记忆落盘 <code>.agent/</code> 目录，密钥经 <code>.env</code> 注入
多渠道接入	同一 Agent 多入口	REST API、SSE 流式、WebSocket、Shell REPL、前端控制台
ReAct 推理	思考→行动→观察迭代	<code>ReActLoopService</code> 驱动 LLM 自主决策工具调用
工具生态	标准化工具接入	内置工具（file/shell/http/memory）+ MCP 协议（stdio/HTTP）+ SPI 扩展
分层记忆	突破上下文窗口	五层记忆（指令→Hot→Summary→Fact→Archive）+ Token 预算 + 检索召回
多 Agent 编排	专家协作	编排 SPI（routing/conversational/delegate）+ 配置与编排分离 + 意图驱动选择
多模型适配	兼容主流 LLM	OpenAI 兼容 API（DeepSeek/通义千问等）+ 流式 SSE 解析 + 独立模型配置
架构质量	高可扩展、可测试	整洁架构 + DDD 分层 + 六边形端口适配器 + 依赖倒置

3. 概要设计

3.1 技术选型

维度	选型	说明
语言 / 框架	Java 8 + Spring Boot 2.7.2	COLA 5.0 架构骨架

维度	选型	说明
构建	Maven 多模块	parent + 6 子模块
LLM 调用	OkHttp + OpenAI 兼容 API	统一 Chat Completions，流式 SSE
流式输出	SseEmitter + TextWebSocketHandler	Token 级实时推送
工具协议	MCP（Model Context Protocol）	stdio / streamable_http
Shell 终端	JLine 3.20	ANSI 着色、命令历史
持久化	本地文件（.agent/ 目录）	会话 JSON + 记忆文件
前端	原生 HTML/CSS/JS	零依赖测试控制台

3.2 模块全景

```
mwb-ai-claw-parent (pom)
├─ mwb-ai-claw-client          # 客户端 SDK: AgentServiceI 接口、ChatCmd、DTO
├─ mwb-ai-claw-adapter        # 适配层: REST/SSE Controller、WebSocket、Shell REPL
├─ mwb-ai-claw-app            # 应用层: AgentServiceImpl + 命令执行器
├─ mwb-ai-claw-domain         # 领域层: 聚合根、领域服务、Gateway 端口接口
├─ mwb-ai-claw-infrastructure # 基础设施层: Gateway 适配器实现、工具、MCP、记忆
└─ start                      # 启动模块: Application、配置文件模板
```

3.3 领域子域划分

子域	职责	核心类型
core 核心域	Agent/Session/Message 聚合、ReAct 引擎、路由	Session、Agent、Message、ReActLoopService、ModelConfig、ReActionResult
collaboration 编排域	多 Agent 协作编排（SPI 插件化 + 协作工具）	OrchestrationDefinition、AgentOrchestrator、ExecutionUnit、CollaborationResult、TodoDefinition、DelegateDefinition、AbstractCollaborationTool
context 上下文工程域	上下文统一组装与消息清洗	ContextAssembler
llm LLM 域	大模型调用抽象（端口）	LlmGateway、EmbeddingGateway、LlmRequest/Response、LlmStreamCallback
tool 工具域	工具执行抽象与安全	ToolGateway、ToolExecutor、ToolSpec、ToolResult
skill 技能域	技能（SKILL.md）发现与按需加载（渐进式披露）	Skill、SkillGateway、UseSkillTool
memory 记忆域	分层记忆与检索	LayeredMemoryGateway、MemoryPage、PageEvictionPolicy、MemoryRetriever

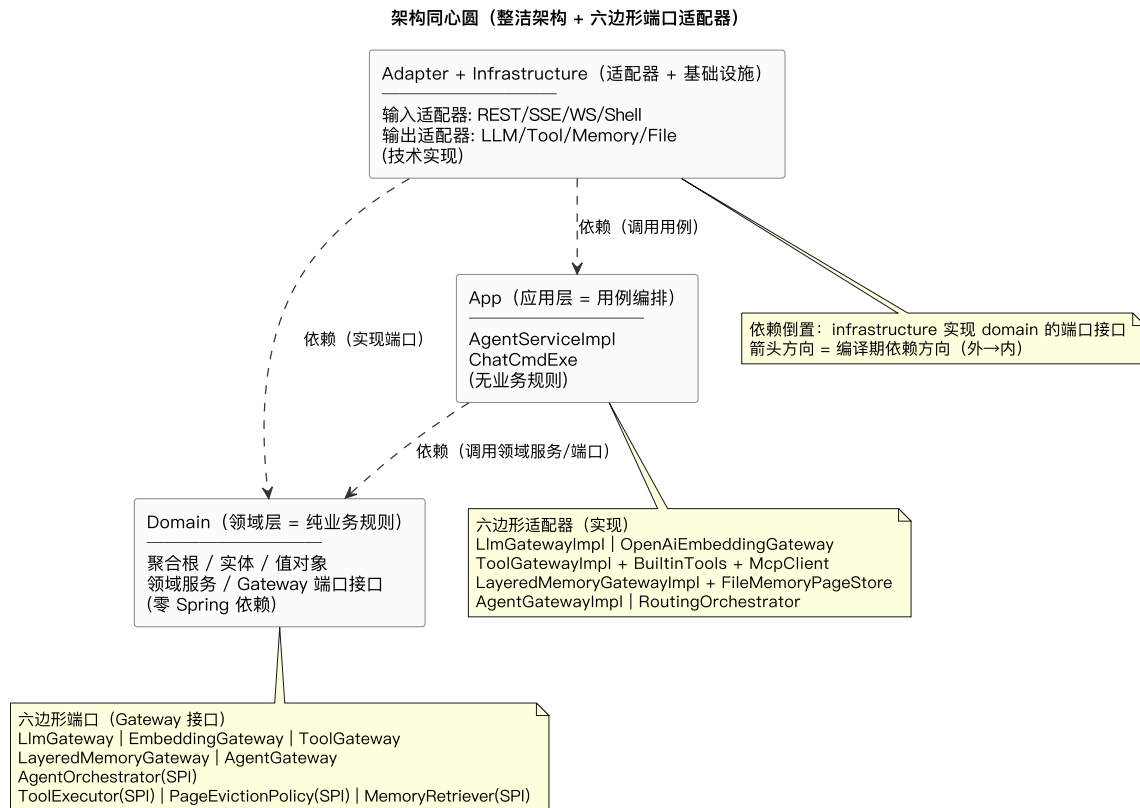
4. 整体架构设计

本项目的架构融合了 整洁架构（Clean Architecture）、领域驱动设计（DDD）和 六边形架构（Hexagonal Architecture）三者的核心理念：

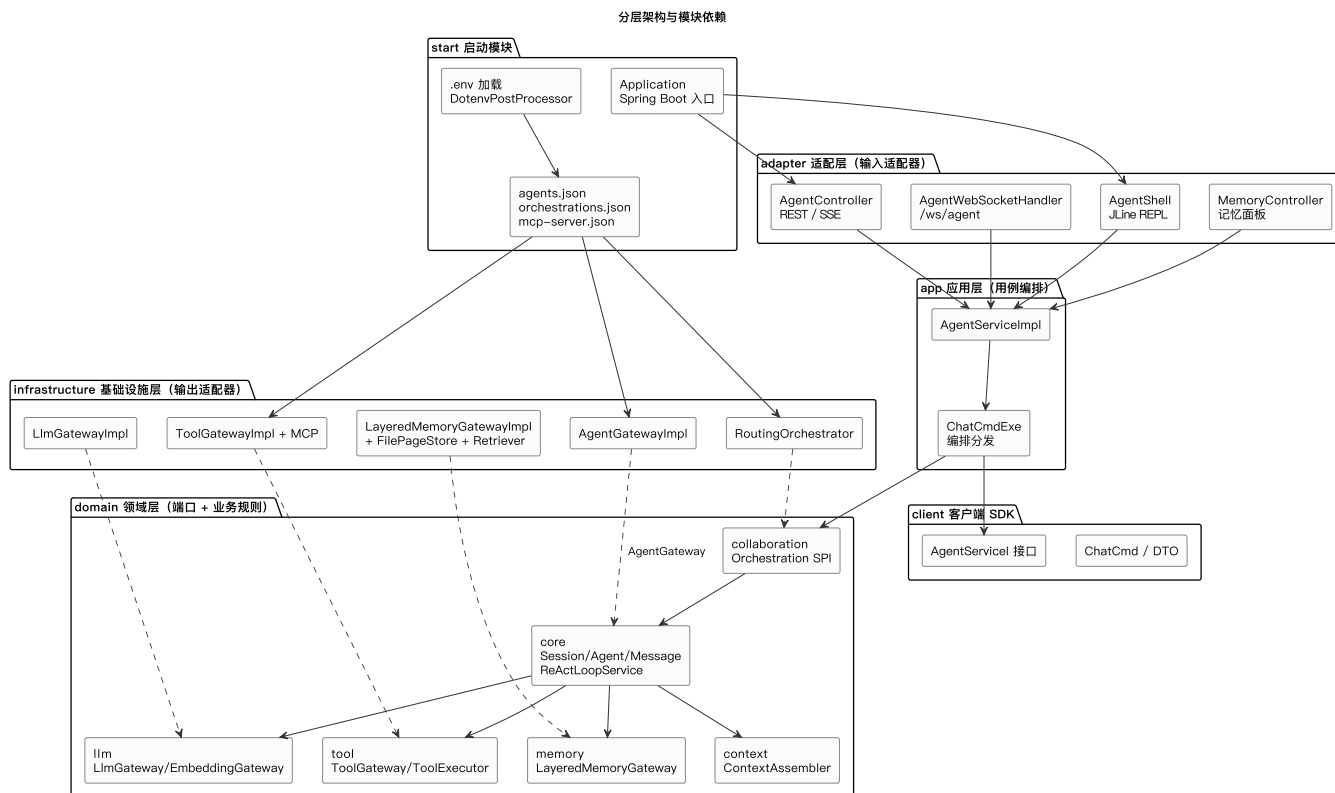
- 整洁架构的「依赖规则」：外层依赖内层，内层不感知外层——domain 是最内层核心，不依赖任何技术框架。
- DDD 的「领域分层 + 聚合根」：将业务拆分为多个限界上下文（子域），核心域持有聚合根（Session），编排域、记忆域等各自自治。

- 六边形架构的「端口与适配器」：domain 层定义 Gateway 接口（端口），infrastructure 层提供实现（适配器），adapter 层作为输入适配器（REST/WS/Shell），三者通过端口解耦。

4.1 架构同心圆（整洁架构 + 六边形端口适配器）



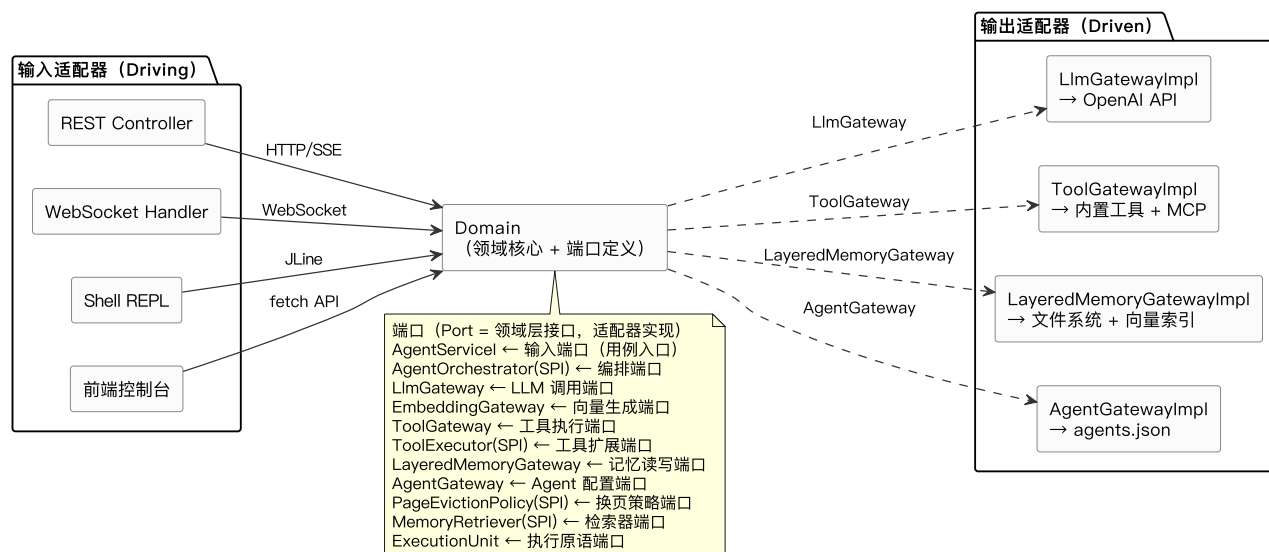
4.2 分层架构与模块依赖



4.3 六边形端口-适配器映射

六边形架构的核心是：领域层定义端口（接口），基础设施层提供适配器（实现），适配器层作为外部世界与领域交互的入口。

六边形端口-适配器映射



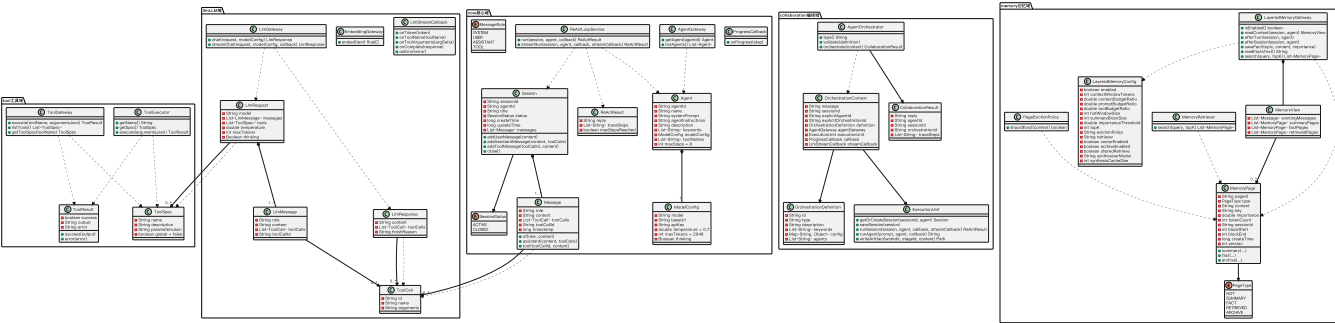
4.4 架构设计原则

原则	体现
依赖规则（整洁架构）	依赖方向严格外→内：adapter/infrastructure → app → client + domain；domain 零 Spring 依赖，可独立测试

原则	体现
依赖倒置 (DIP)	domain 定义 Gateway 接口 (端口), infrastructure 实现 (适配器), 编译期 domain 不依赖 infrastructure
聚合根 (DDD)	Session 是核心域聚合根, 所有消息操作经 Session 的 addUserMessage / addAssistantMessage / addToolMessage 方法
限界上下文 (DDD)	6 个子域各自自治: core (会话+推理)、collaboration (编排)、memory (记忆)、llm (模型)、tool (工具)、context (上下文)
端口适配器 (六边形)	输入适配器 (REST/WS/Shell) → 用例端口 (AgentServiceI) → 领域 → 输出适配器 (LLM/Tool/Memory Gateway 实现)
SPI 可插拔	AgentOrchestrator、ToolExecutor、PageEvictionPolicy、MemoryRetriever 均为 SPI 接口, Spring 自动收集注册
配置分离	Agent 注册表 (agents.json) 与编排注册表 (orchestrations.json) 解耦, 编排插件 SPI 与选择逻辑分离

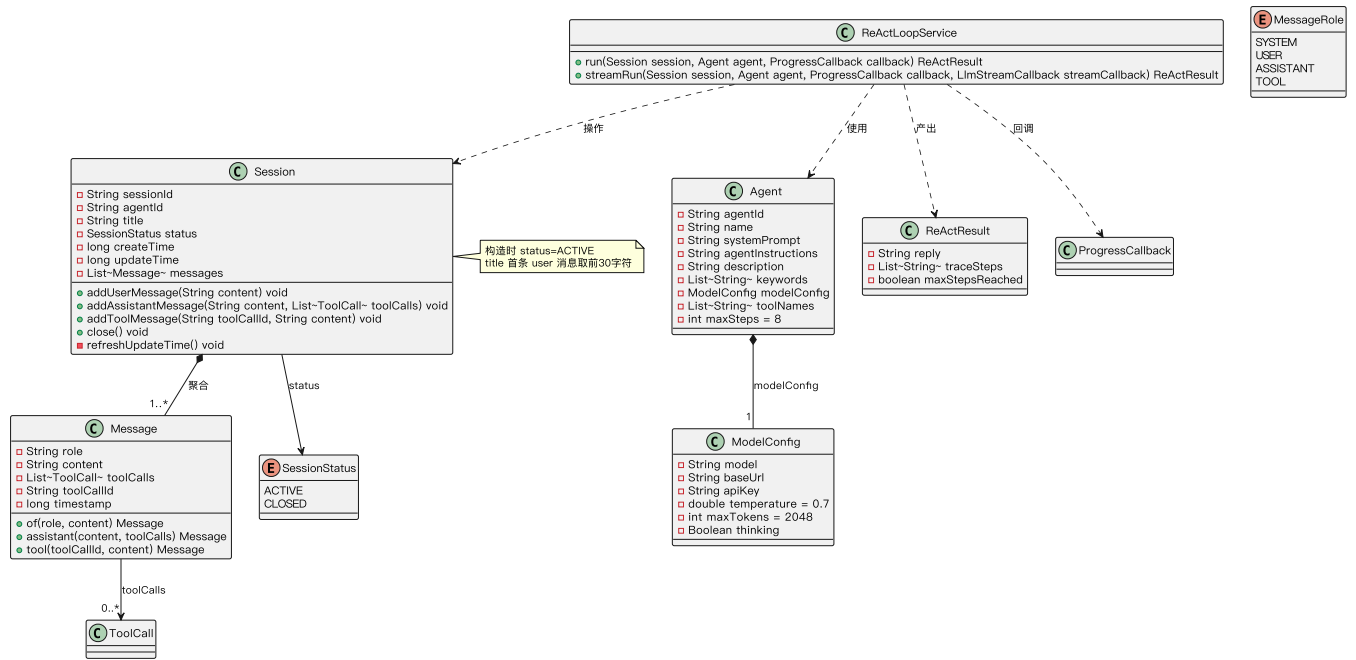
5. 数据模型设计

5.1 领域模型全景图

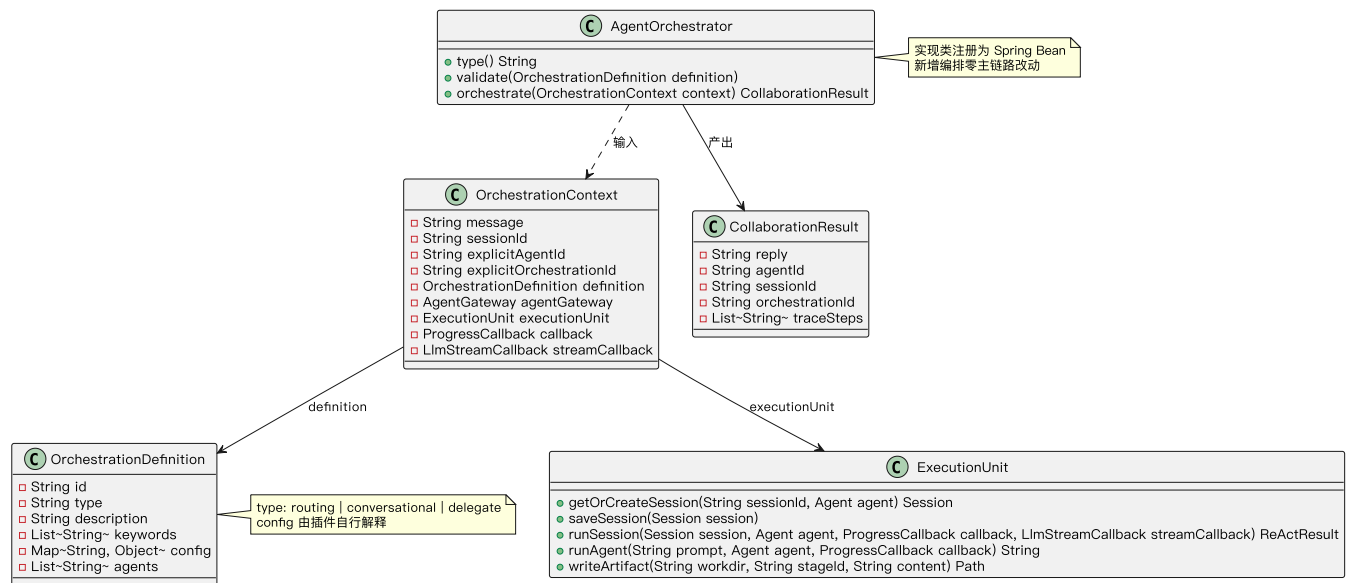


5.2 核心域数据模型

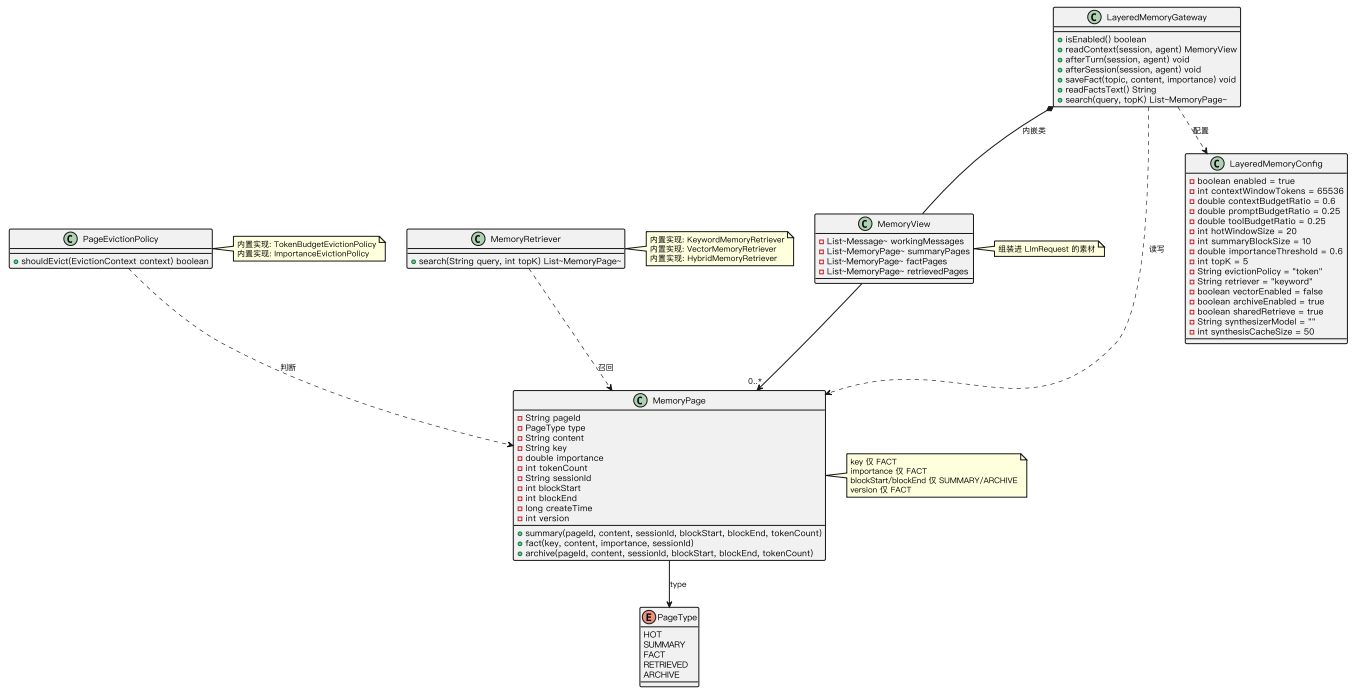
核心域是整个系统的业务核心, 包含 Session 聚合根、Agent 实体、Message 实体及 ReAct 推理引擎。



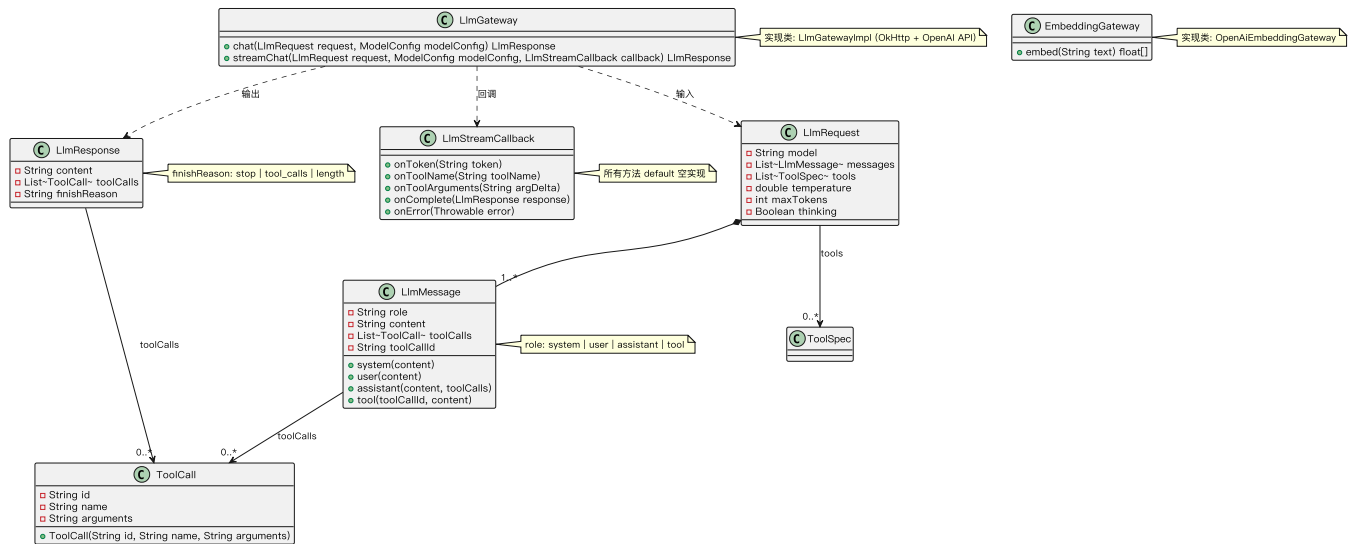
5.3 编排协作域数据模型



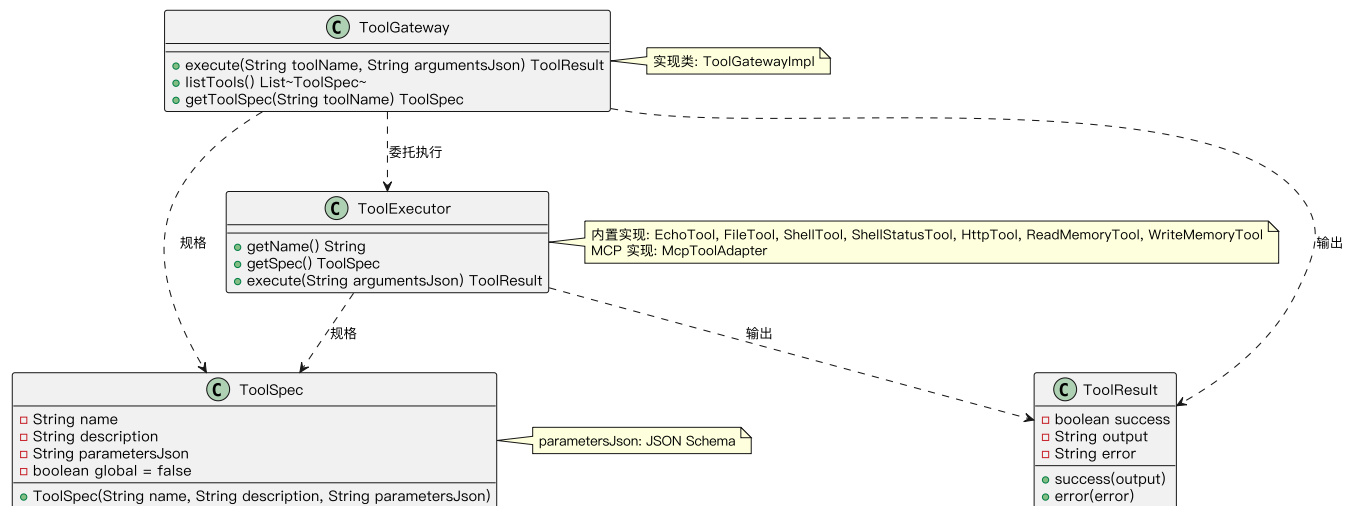
5.4 记忆域数据模型



5.5 LLM 域数据模型



5.6 工具域数据模型



5.7 枚举定义汇总

枚举	所属域	值	用途
SessionStatus	core	ACTIVE , CLOSED	会话生命周期状态
MessageRole	core	SYSTEM , USER , ASSISTANT , TOOL	消息角色（已废弃： Message.role 改用 String, 枚举无引用）
PageType	memory	HOT , SUMMARY , FACT , RETREVED , ARCHIVE	记忆页类型

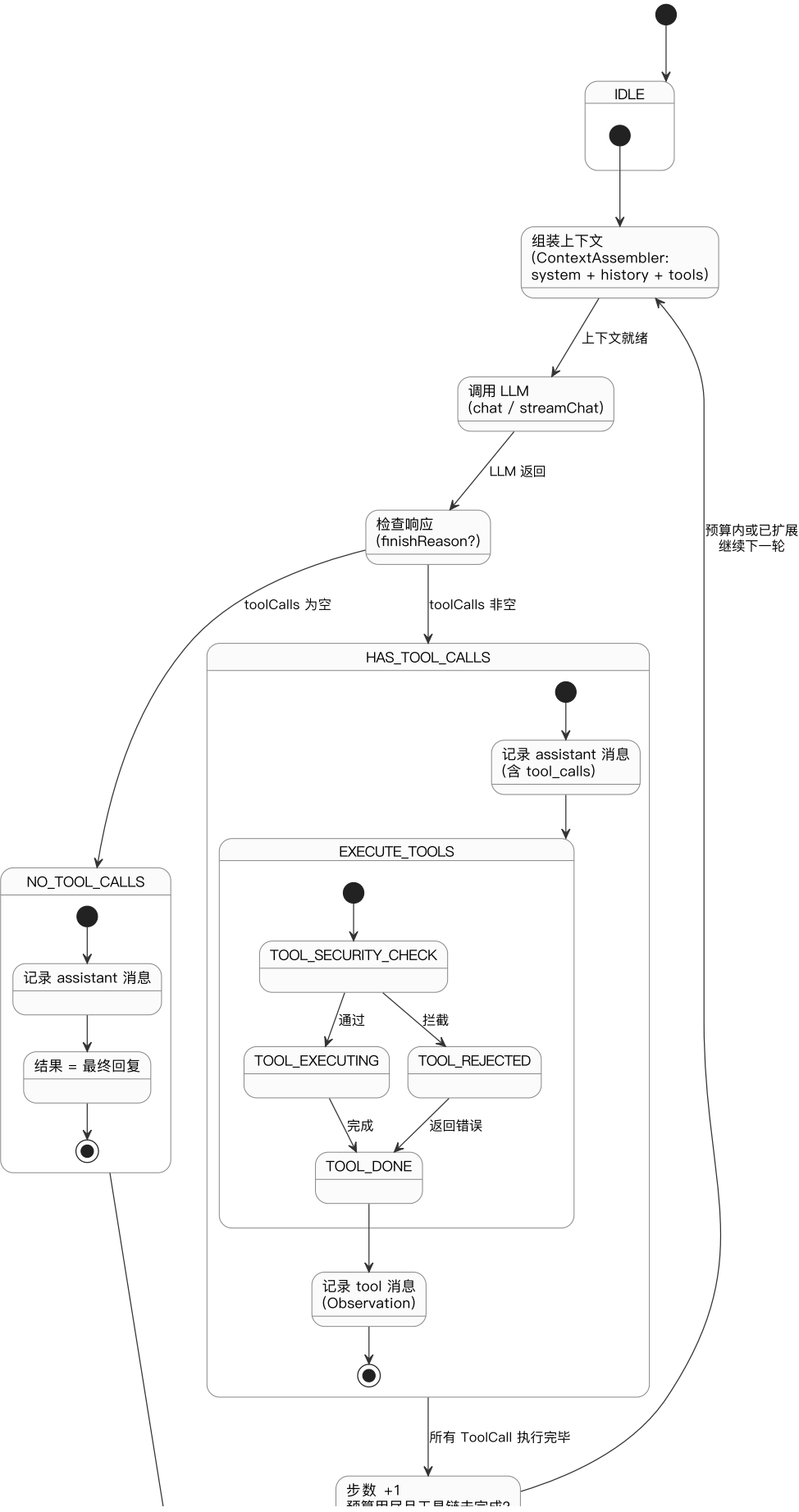
6. 重要领域的详细流程设计

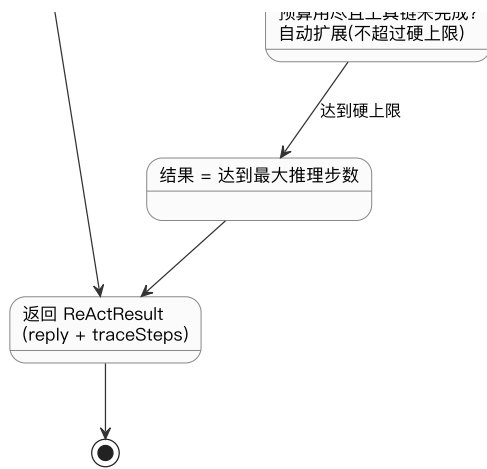
6.1 核心域：ReAct 推理引擎

6.1.1 ReAct 循环状态图

ReAct 推理引擎是整个系统的核心，驱动「思考→行动→观察」迭代循环。每轮循环中，LLM 要么直接产出最终回复（终止），要么声明工具调用（进入工具执行后继续循环）。

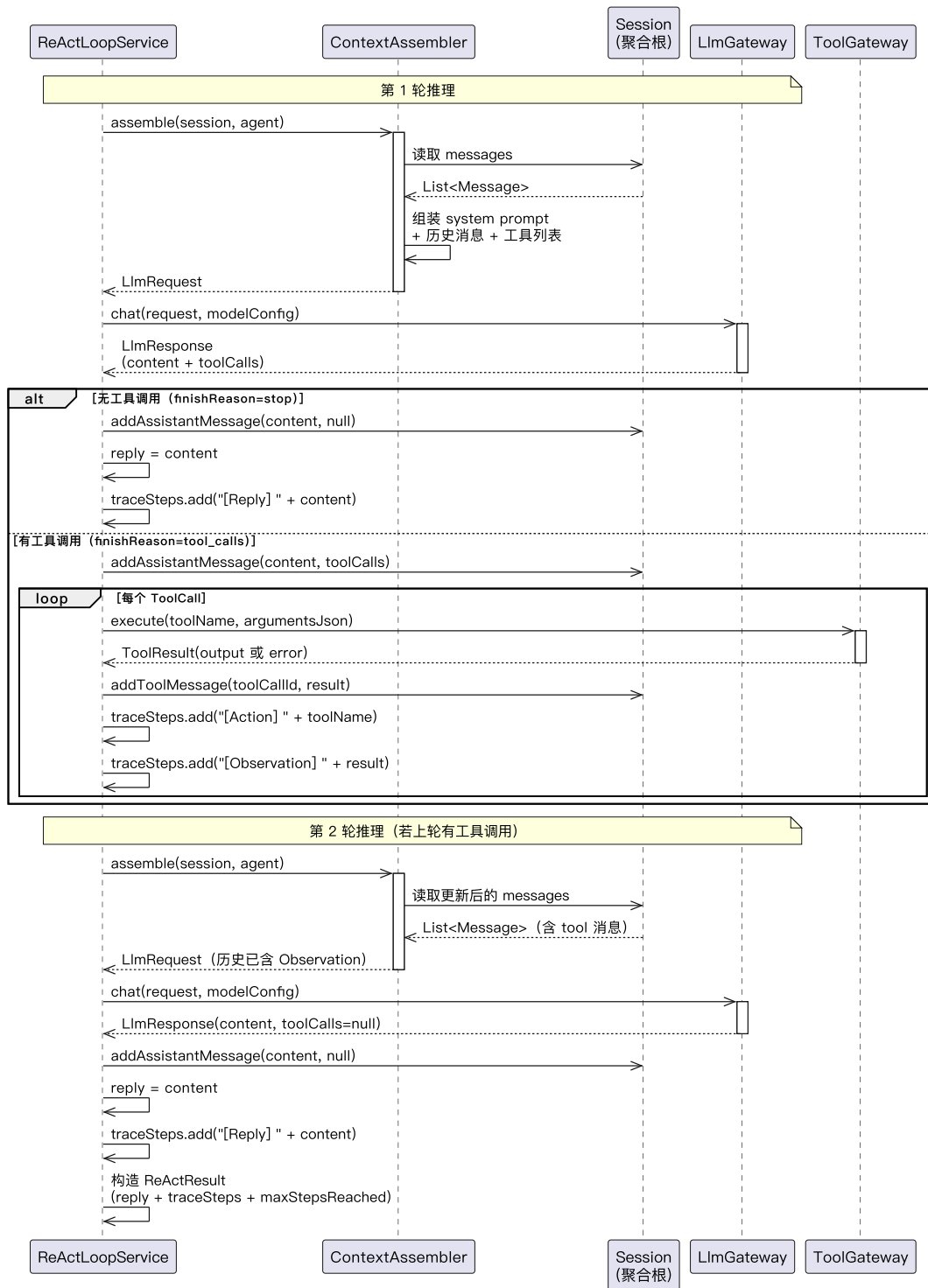
ReAct 循环状态图





6.1.2 ReAct 循环数据交互图

ReAct 循环数据交互图



6.1.3 文字说明

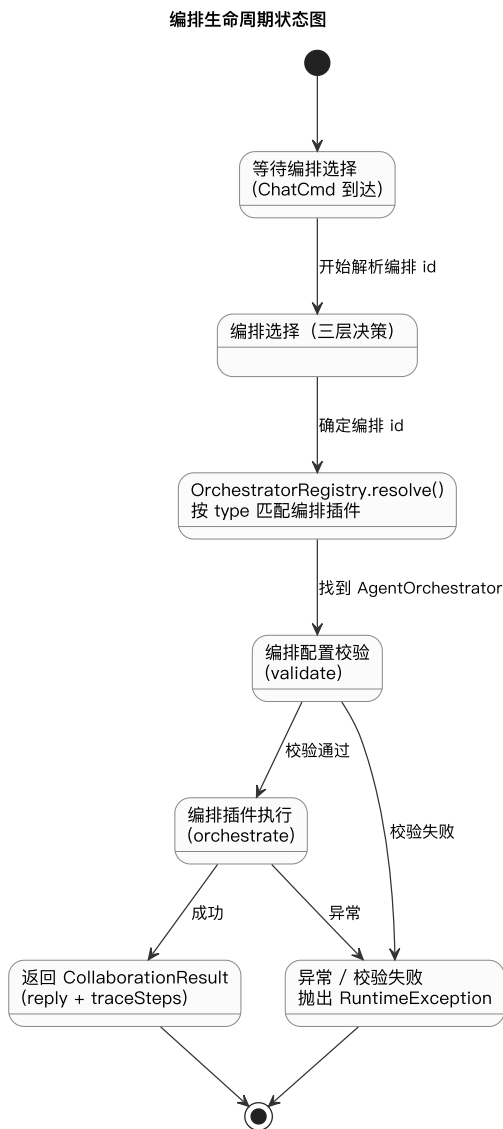
ReAct (Reasoning + Acting) 循环由 [ReActLoopService](file:///Users/mawenbin/workspace/java/mwb_coding/mwb-ai-claw/mwb-ai-claw-domain/src/main/java/com/mwb/ai/claw/domain/core/ReActLoopService.java) 驱动，核心流程如下：

- 上下文组装：**每轮推理开始，调用 `ContextAssembler.assemble()` 组装 LLM 请求。组装内容包括 system prompt (Agent 人设 + [AGENT.md](#) 指令 + 长期记忆 + **推理预算提示**：告知本次任务预算步数，引导并行调用相互独立的工具、获得足够信息后直接给出最终回复)、会话历史消息（或分层记忆的工作记忆视图）、工具规格列表。
- LLM 调用：**通过 `LlmGateway` 发送请求。支持同步（`chat`）与流式（`streamChat`）两种模式。流式模式下通过 `LlmStreamCallback` 逐 token 推送增量内容。

3. 响应判定：检查 `LlmResponse.finishReason` ——若为 `stop` 且无 `toolCalls`，则 LLM 已产出最终回复，循环终止；若为 `tool_calls`，则进入工具执行阶段。
4. 工具执行：遍历 LLM 声明的 `ToolCall` 列表，逐个调用 `ToolGateway.execute()` 执行。每个工具结果作为 `tool` 角色消息 (Observation) 追加到会话。
5. 迭代或终止（软预算 + 自动扩展）：工具执行完毕后步数 +1。初始预算为 `Agent.maxSteps`（默认 8），硬上限 = `maxSteps × maxStepsExtensionFactor`（默认 2.0）。预算用尽但本轮仍调用了工具（工具链未收敛）时，自动追加 `max(maxSteps/2, 1)` 步继续推理（每次扩展在 `traceSteps` 记录 [Info] 步数预算已用尽...自动扩展）；达到硬上限仍未产出最终回复，才以「达到最大推理步数」终止。
6. 结果返回：构造 `ReActResult`，包含最终回复 `reply`、执行轨迹 `traceSteps` (Thought/Action/Observation 摘要)、以及 `maxStepsReached` 标志。

6.2 编排协作域

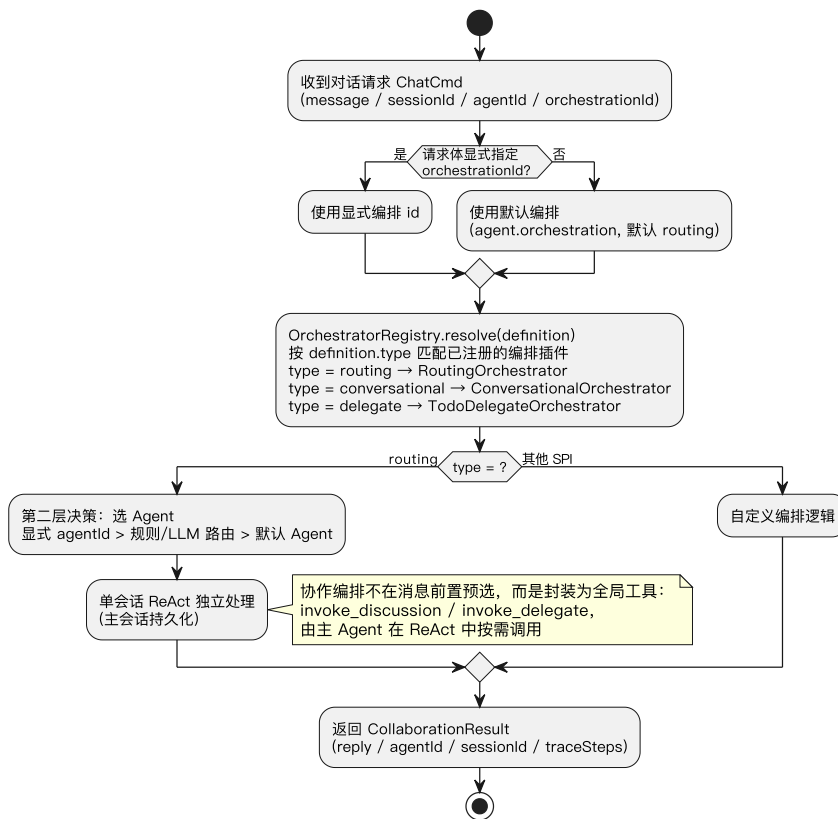
6.2.1 编排生命周期状态图



6.2.2 编排选择（协作工具化）

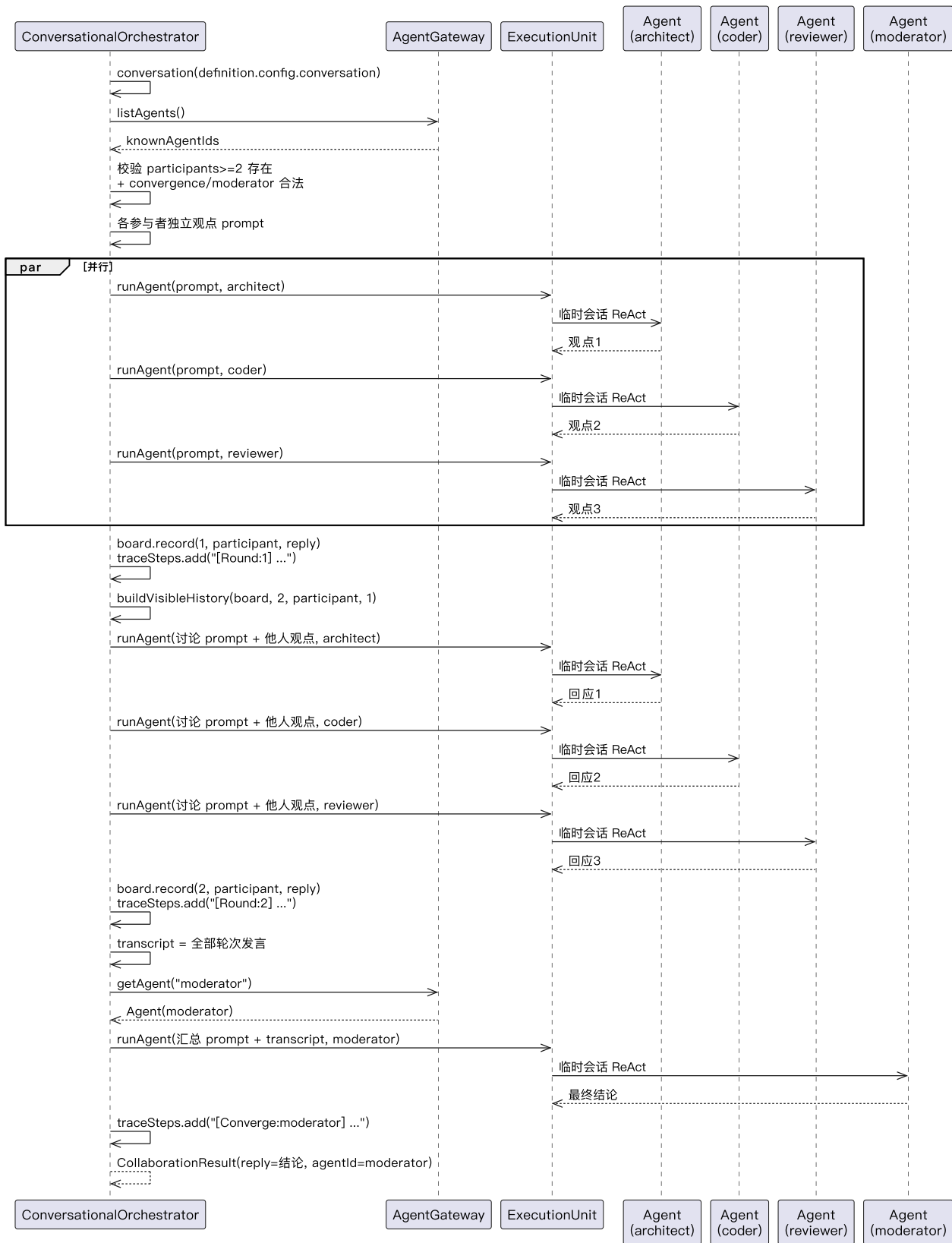
对话请求默认走 `routing` 编排（单专家 ReAct 独立处理）；多 Agent 协作编排（对话式 / 委托）不再做消息前置意图路由，而是封装为全局协作工具（`invoke_discussion` / `invoke_delegate`，基于 `ExecutionUnit.runOrchestration`），由主 Agent 在 ReAct 推理中自主决定是否发起。编排选择简化为两层：显式指定（`orchestrationId`）> 默认编排（`agent.orchestration`，默认 `routing`）。

编排选择活动图（协作工具化）



6.2.3 对话式编排数据交互图

对话式编排数据交互图



6.2.4 文字说明

编排协作域实现多 Agent 协作，核心设计是 **SPI 插件化 + 主 Agent 主导协作**：

编排选择（简化两层）：

1. **第一层（选编排）**：ChatCmdExe 根据 `orchestrationId` 决定使用哪个编排。优先级：显式指定 > 默认编排（`agent.orchestration`，默认 `routing`）。多 Agent 协作编排不再消息前置预选，而是封装为全局工具（`invoke_discussion` / `invoke_delegate`），由主 Agent 在 ReAct 中按需调用（经 `ExecutionUnit.runOrchestration` 嵌套调起）。
2. **第二层（选 Agent）**：编排插件内部决定使用哪个 Agent。Routing 编排通过显式 `agentId` / 规则关键词 / LLM 路由选择；协作编排（对话式 / 委托式）由各自定义中的 `agentId` 字段指定。
3. **第三层（选工具）**：Agent 的 `toolNames` 配置 + 全局工具（协作工具 / MCP 动态注册）决定可用工具集。

编排 SPI（`AgentOrchestrator`）：

- 实现类通过 `type()` 声明编排类型标识，注册中心在启动期自动收集。
- 新增编排方式仅需：(1) 实现接口并注册为 Spring Bean；(2) 在 `orchestrations.json` 增加一条定义。
- 内置实现：`RoutingOrchestrator`（路由，单 Agent 独立处理）、`ConversationalOrchestrator`（对话式，多方多轮讨论 + 收敛）、`TodoDelegateOrchestrator`（委托式，规划 Todo → 委派子 Agent，可递归 + 并行）。

对话式编排（`ConversationalOrchestrator`）：

- 定义（`ConversationDefinition`）从编排定义的 `config.conversation` Map 解析：`rounds` / `participants` / `moderator` / `convergence` / `minConsensus` / `visibleHistory` / `thinking`。
- 执行流程：首轮（并行，`CompletableFuture` + 线程池）各参与者独立产出观点 → 讨论轮（串行）按 `visibleHistory` 截断注入其他参与者发言并互相回应 → 收敛产出最终结论。
- 收敛策略：`moderator`（默认，仲裁 Agent 汇总全部发言）、`consensus`（发言含共识信号词数 $\geq$ `minConsensus` 提前终止，取支持最多的发言）、`best`（解析「置信度: 0.x」标注取最高）。
- 参与者与收敛 Agent 均通过临时会话执行（上下文隔离，不入库）；`thinking` 覆盖与空回复重试同委托编排。

委托编排（`TodoDelegateOrchestrator`）：

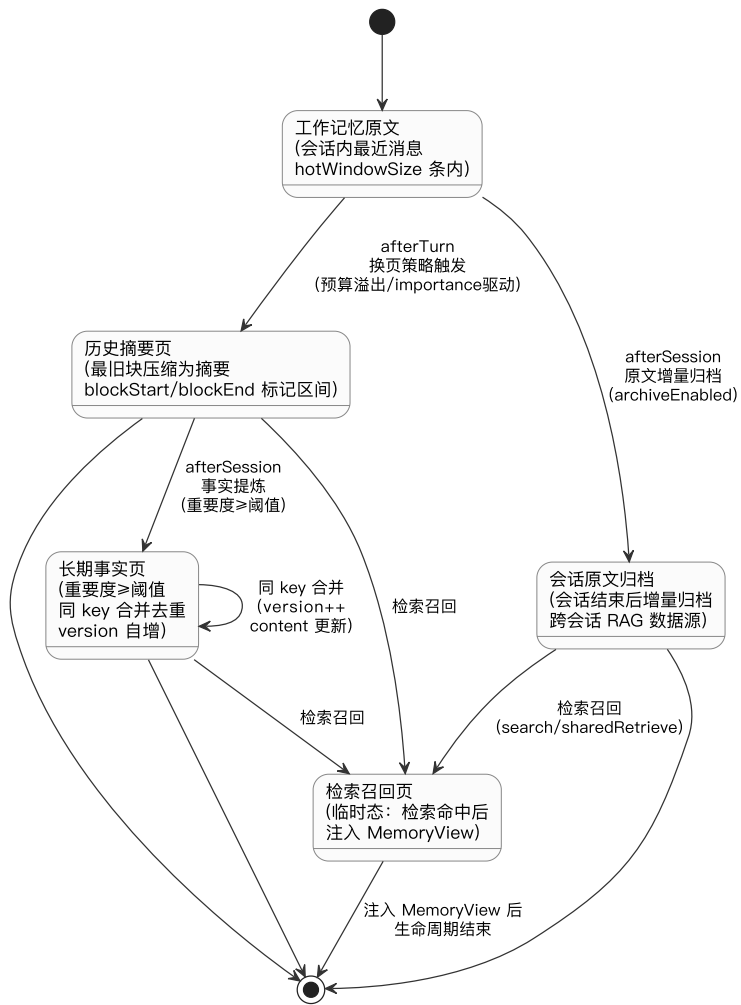
- 定义（`DelegateDefinition`）从编排定义的 `config.delegate` Map 解析：`plannerAgentId` / `maxTodos` / `maxDepth` / `parallel` / `concurrency` / `onFailure` / `retries` / `thinking` / `resultPass` / `workdir`。
- 执行流程：规划（**Plan**）→ 主 Agent 思考并输出 Todo 列表（结构化 JSON：`todoId` / `title` / `description` / `agentId` / `dependsOn`）→ 委派（**Execute**）→ 对 Todo 做 Kahn 拓扑分层，无依赖组为 Wave 并行执行（`CompletableFuture` + 线程池），依赖 Todo 分层推进、结果注入下游 prompt → 汇总（**Summarize**）→ 各 Todo 结果（截断或落盘）交给规划 Agent 输出整体结论。
- 递归委托：子 Agent 执行 Todo 时若 `depth + 1 < maxDepth` 可再规划子 Todo 并委托下一级（任务树），到达深度上限的节点直接执行。
- 容错：规划输出非 JSON 重试一次后降级「直执行」；依赖环检测回退声明顺序串行；todo 失败按 `onFailure: abort / skip` 处理；空回复复用「请直接输出完整回答」重试；未知 `agentId` 回退默认 Agent。
- 约束：`maxDepth` / `maxTodos` 硬限制；`concurrency` 控制并行度（默认 4）；线程池每次编排实例级创建与释放。
- 数据底座：规划 / 汇总产物按 `{workdir}/{sessionId}/{时间戳}` 隔离落盘（`plan-{layerPath}.json` / `result-{layerPath}.txt`），叶子 todo 结论经 `LayeredMemoryGateway.saveFact` 沉淀 FACT（topic 含层级路径幂等去重），落盘与沉淀全程追加轨迹。
- 交互与上下文（P1）：Todo 生命周期状态机（`TodoStatus`：`paused` → `approved` → `running` → `done / failed`）；人工审批门禁 `approvalGate`（`none` / `root` / `all`，命中层规划完成后经 `ApprovalRegistry` 注册并暂停等待，REST / WebSocket 审批 API 或 Shell 命令（`/pending [sessionId]` 查询、`/approve <layerKey> [sessionId]` 批准、`/reject <layerKey> [sessionId]` 拒绝；shell 普通对话改后台线程执行，等待审批期间 REPL 仍可接收决策命令）按 `{sessionId}/{layerKey}` `approve / reject`，拒绝或超时降级直执行）；汇总阶段子结果按与父任务相关性 bigram 打分取 top-k（默认 3）注入，控制上下文成本。
- 智能与组合（P2）：动态规划 `replanRounds`（默认 0 不启用；每个 Wave 执行完成后规划者结合已得结果对剩余 Todo 做 re-plan，支持完整 todos 替换或 `adjust keep/drop/modify` 增量协议，仅作用于未执行 Wave，调整后写 `plan-*.json` 并输出 [Replan] 轨迹）；Todo 级编排嵌套组合（`TodoDefinition.orchestrationId` 可引用 `conversational / delegate` 自身，经 `ExecutionUnit.runOrchestration` 按 id 调起，嵌套 `CollaborationResult.reply` 回传参与本层汇总；防环用线程级嵌套调用链 `ThreadLocal<Deque>` 检测，A→B→A 循环引用运行时抛业务异常终止）。

6.3 记忆域

6.3.1 记忆页生命周期状态图

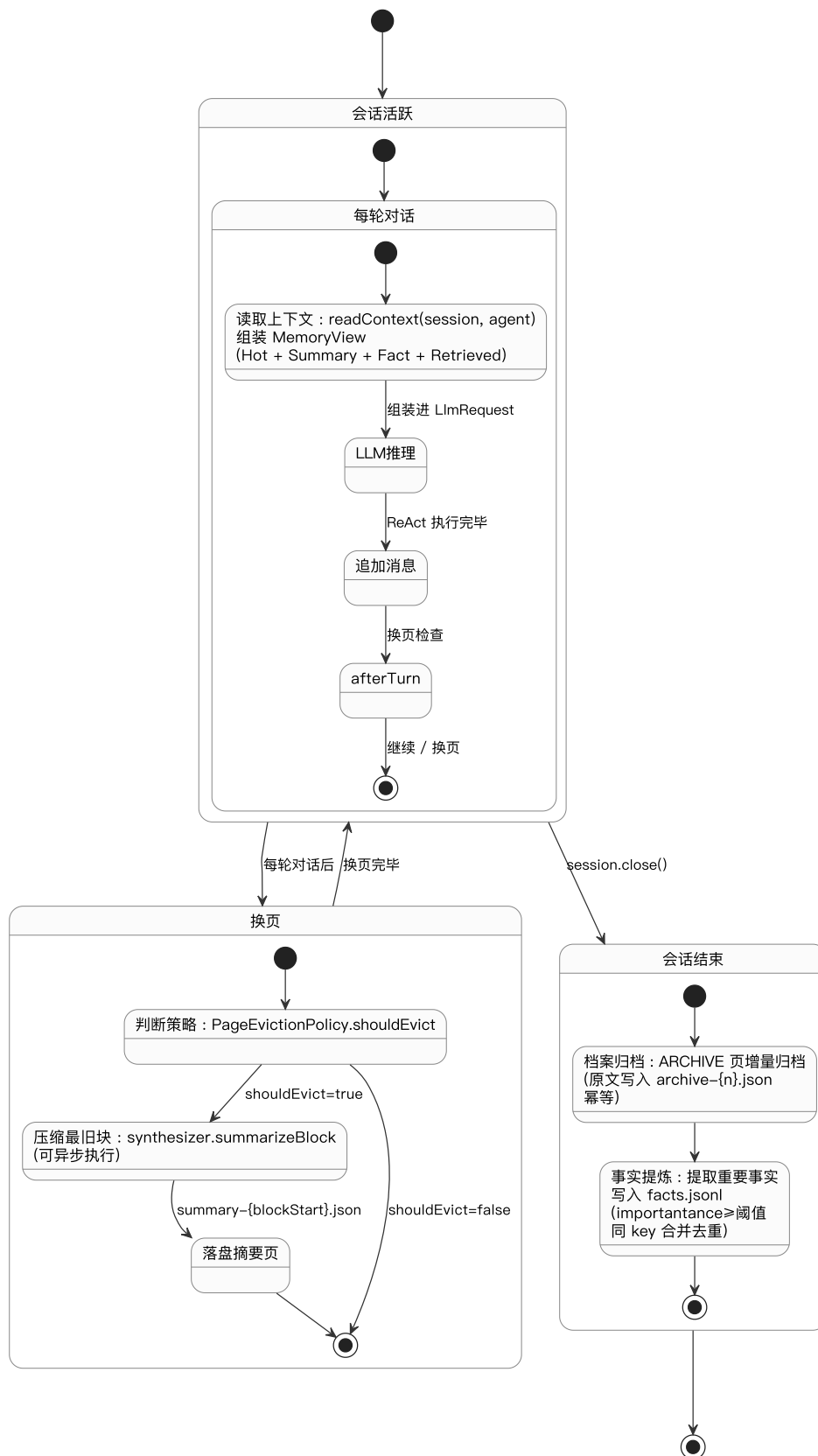
`MemoryPage` 在分层记忆系统中经历多种类型转换：

记忆页生命周期状态图



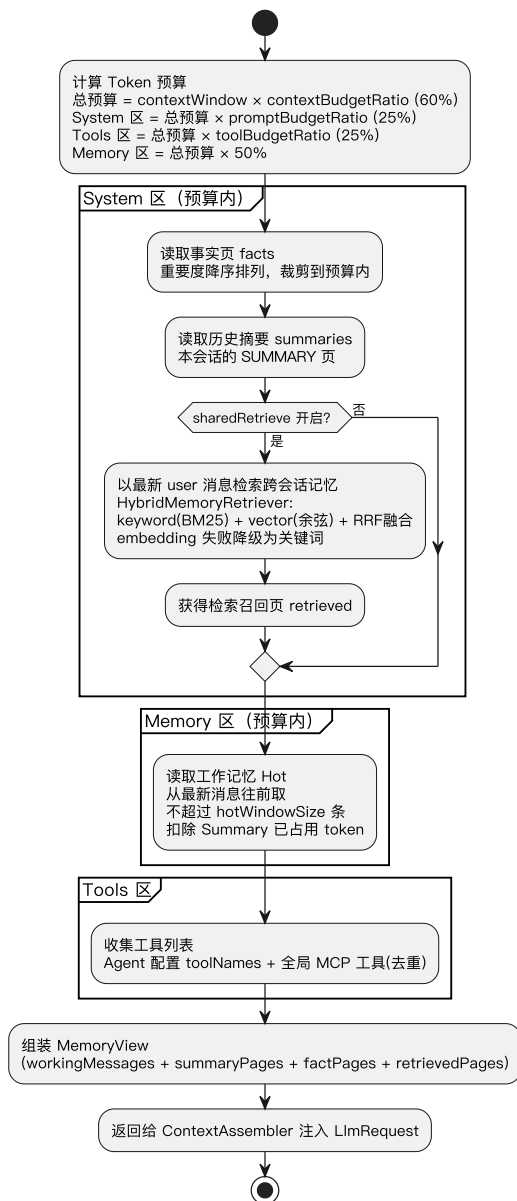
6.3.2 会话记忆流转状态图

会话记忆流转状态图



6.3.3 分层记忆上下文组装活动图

分层记忆上下文组装活动图



6.3.4 文字说明

记忆域实现五层分层记忆模型，核心目标是在有限的 LLM 上下文窗口内，最大化保留有价值的上下文信息：

五层记忆模型：

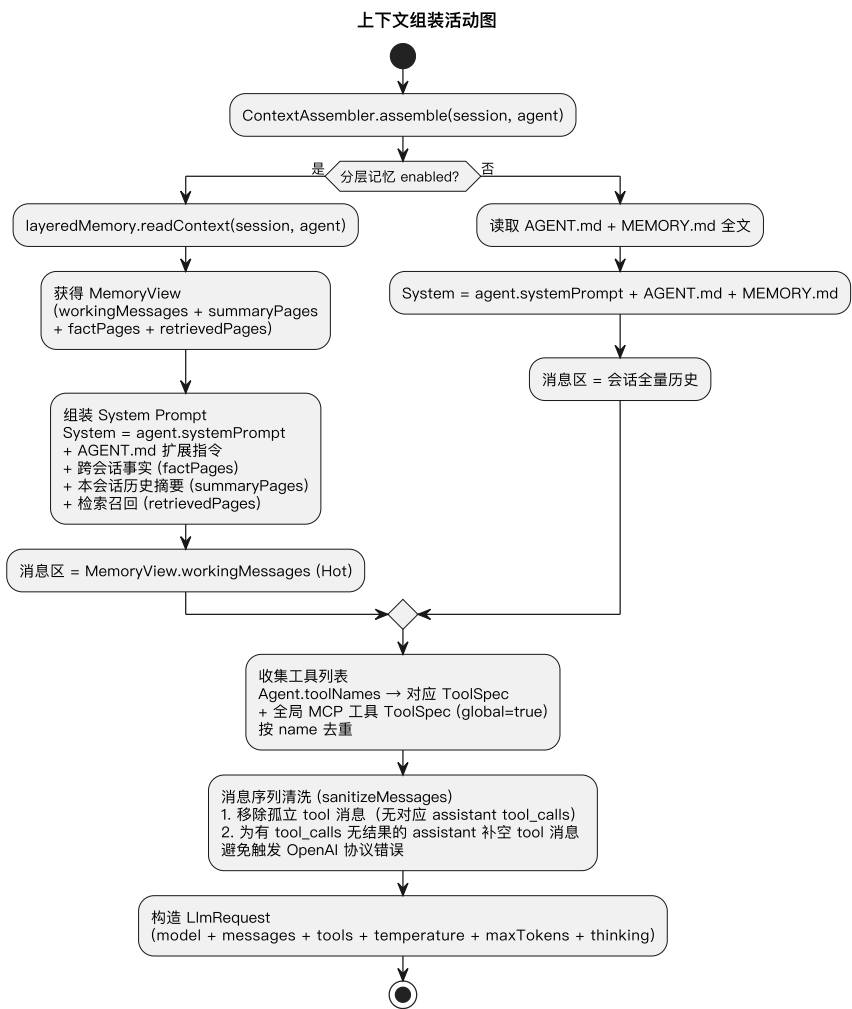
层	类型	内容	存储
指令层	—	AGENT.md 扩展指令	.agent/AGENT.md
工作记忆	HOT	预算内最近原文消息	会话内存
短期	—	会话全量原文	.agent/sessions/*.json
中期	SUMMARY	块摘要页	.agent/memory/pages/{sessionId}/summary-*.json
长期	FACT	结构化事实（重要度过滤 + 合并去重）	.agent/memory/facts.jsonl
档案	ARCHIVE	会话原文归档（跨会话 RAG 数据源）	.agent/memory/pages/{sessionId}/archive-*.json

核心机制：

- **Token 预算驱动**：总预算 = `contextWindow × contextBudgetRatio(0.6)`，按 System(25%)/Tools(25%)/Memory(50%) 分配，各区域独立裁剪。
- **换页策略 (SPI)**：PageEvictionPolicy 接口，内置 TokenBudgetEvictionPolicy（默认，预算溢出触发）和 ImportanceEvictionPolicy（重要度驱动，低价值提前压缩）。
- **异步提炼**：摘要/事实提炼经 MemorySynthesisExecutor 独立线程池串行执行，不阻塞主对话链路。结果按内容哈希经 SynthesisCache 去重缓存。
- **成本优化**：synthesizerModel 独立小模型提炼，降低成本。
- **检索召回 (SPI)**：MemoryRetriever 接口，内置 KeywordMemoryRetriever（BM25 关键词）、VectorMemoryRetriever（向量余弦）、HybridMemoryRetriever（RRF 融合）。embedding 失败自动降级为关键词检索。
- **跨会话共享**：sharedRetrieve 开启时，以最新 user 消息跨会话检索事实/摘要/档案，实现多 Agent 共享记忆。
- **消息清洗**：sanitizeMessages 双向兜底，避免「孤立 tool 消息」或「有 tool_calls 无结果的 assistant」导致的 OpenAI 协议错误。

6.4 上下文工程域

6.4.1 上下文组装活动图



6.4.2 文字说明

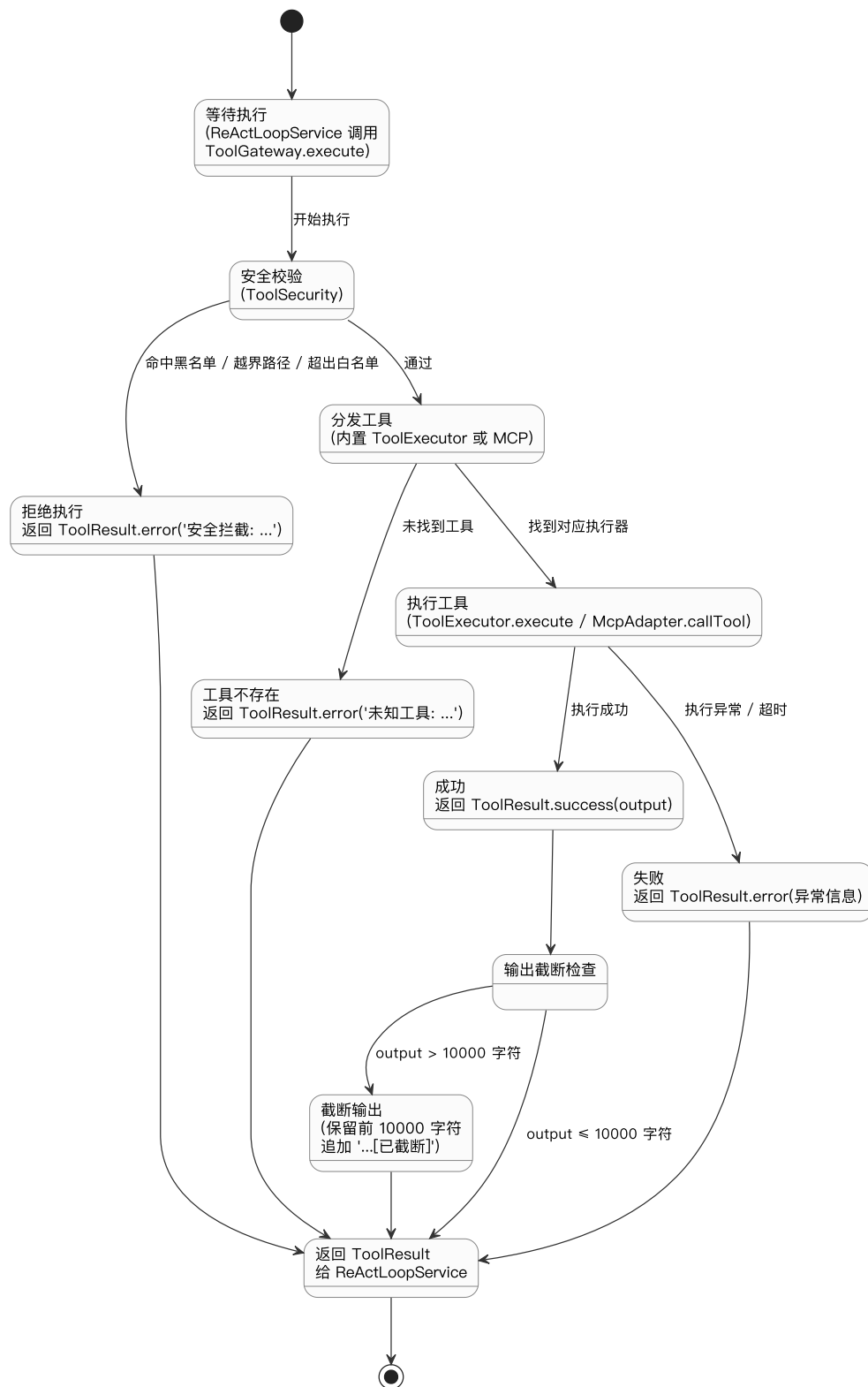
上下文工程域由 [ContextAssembler](file:///Users/mawenbin/workspace/java/mwb_coding/mwb-ai-claw/mwb-ai-claw-domain/src/main/java/com/mwb/ai/claw/domain/context/DefaultContextAssembler.java) 实现，是 ReAct 循环与 LLM/记忆/工具之间的桥梁：

1. **System Prompt 组装**: Agent 人设 (`systemPrompt`) → [AGENT.md](#) 扩展指令 → 长期记忆事实 → 历史摘要 → 检索召回, 按序拼接。
 2. **消息区组装**: 分层记忆开启时使用 `MemoryView.workingMessages` (预算内最近 Hot 消息); 未开启时使用会话全量历史。
 3. **工具列表组装**: Agent 显式配置的 `toolNames` 对应的 `ToolSpec` + 全局 MCP 工具 (`global=true`), 按 name 去重。
 4. **消息序列清洗**: `sanitizeMessages` 双向兜底——移除孤立的 tool 消息 (无对应 assistant tool_calls), 为有 tool_calls 但缺结果的 assistant 补充空 tool 消息, 确保消息序列符合 OpenAI Chat Completions 协议要求。
 5. **模型参数注入**: 将 `ModelConfig` 中的 `temperature` 、 `maxTokens` 、 `thinking` 注入 `LlmRequest` 。
-

6.5 工具域

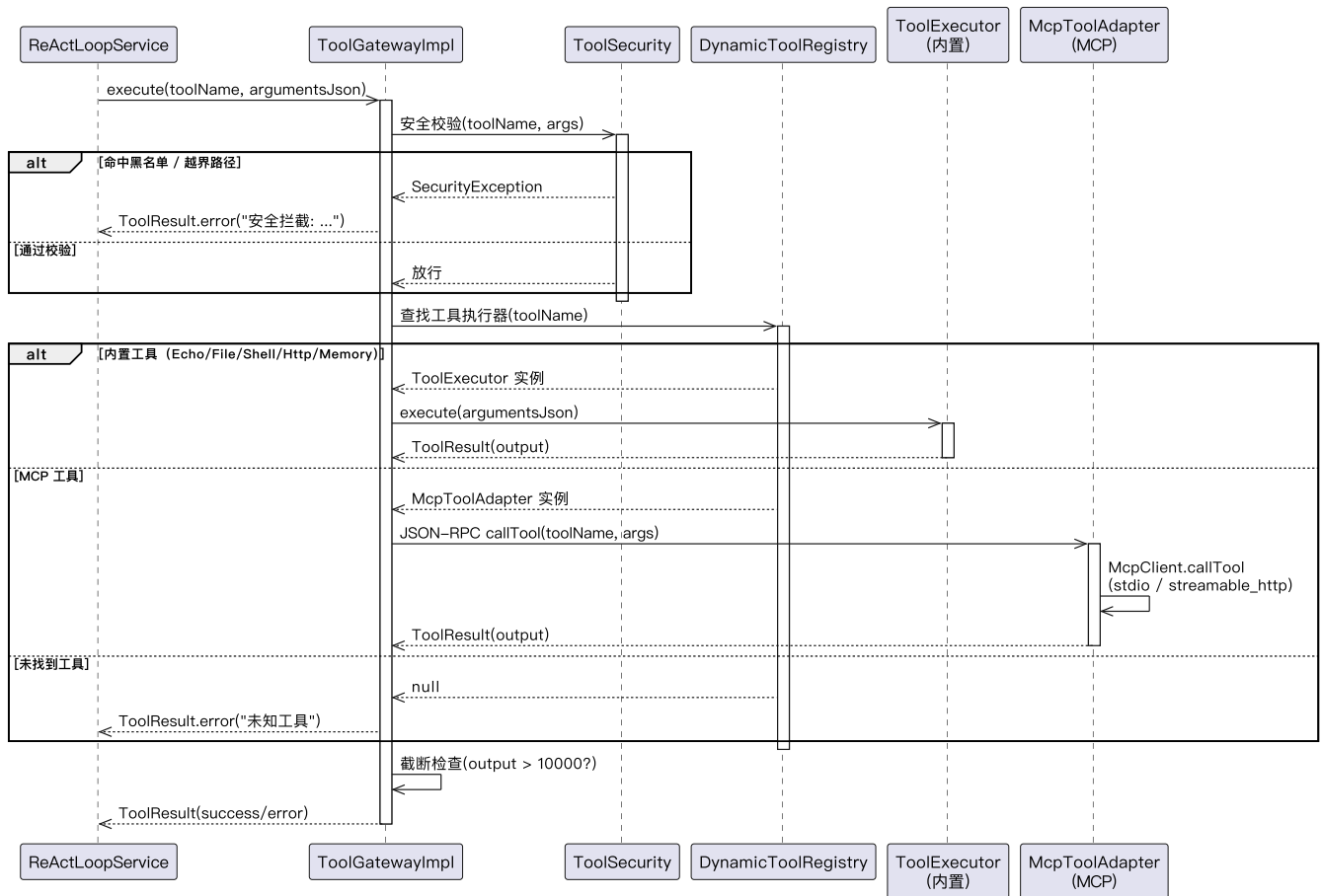
6.5.1 工具执行状态图

工具执行状态图



6.5.2 工具调用数据交互图

工具调用数据交互图



6.5.3 文字说明

工具域提供 Agent 执行外部操作的能力，核心是 **SPI 扩展 + 安全沙箱 + MCP 协议**：

工具 SPI (ToolExecutor)：

- 新增工具只需实现 ToolExecutor 接口并注册为 Spring Bean， ToolGatewayImpl 通过 DynamicToolRegistry 自动收集。
- 内置工具： echo（测试）、file（文件读写）、shell（命令执行）、shell_status（后台任务查询/终止）、http（网络请求）、read_memory/write_memory（长期记忆读写）。
- 流式执行： ToolExecutor / ToolGateway 提供回调版 execute(argumentsJson, ProgressCallback)，Shell 命令输出逐行推送（ [Stream] ），供终端实时回显。
- MCP 工具：通过 McpToolRegistrar 将 MCP Server 暴露的工具动态注册为全局工具（ global=true ），对所有 Agent 可见。

安全沙箱 (ToolSecurity)：

- Shell 语义：经 bash -lc（Windows 为 cmd /c）执行，完整支持管道 / 重定向 / 通配符 / && / 环境变量。
- 命令白名单：允许的 Shell 命令，按命令段逐段校验（ splitShellSegments 引号感知切分，防 ls; rm -rf / && 拼接绕过）。
- 命令黑名单：21 个危险模式（ rm -rf / 、 sudo 、 mkfs 、 fork bomb 等），优先级高于白名单。
- 审批模式： shell-approval-mode 三档（ auto 自动执行 / ask 命中规则弹 Y/N 确认（默认） / read-only 拒绝）， shell-approval-patterns 配置 50+ 高风险规则（ git push 、 rm 、 npm install 、 npm cache 、 find -delete 、 curl -X 等）； ToolApproval 领域接口由 Shell REPL 实现， headless / 无审批器场景安全默认拒绝。
- 长时任务：前台超时不再强杀，转为后台任务（ ShellProcessManager ）返回 taskId， shell_status 工具（ status/output/kill ）查询 / 终止； shell 支持 background=true 参数。
- 路径限制：File/Shell 工具仅允许在配置的 workspace-dir 内操作。
- 输出截断：工具输出限制 10000 字符（截断前先脱敏）。
- 敏感信息脱敏：shell 输出与工具入参中的密钥（ sk- / api_key= / token: / password= / Bearer / AKIA ）经 maskSecrets 打码后再进上下文。
- HTTP 限制：可配置 http-allowed-hosts 防 SSRF。

- 所有安全违规捕获为 `SecurityException`，返回 `ToolResult.error`，不中断 ReAct 循环。

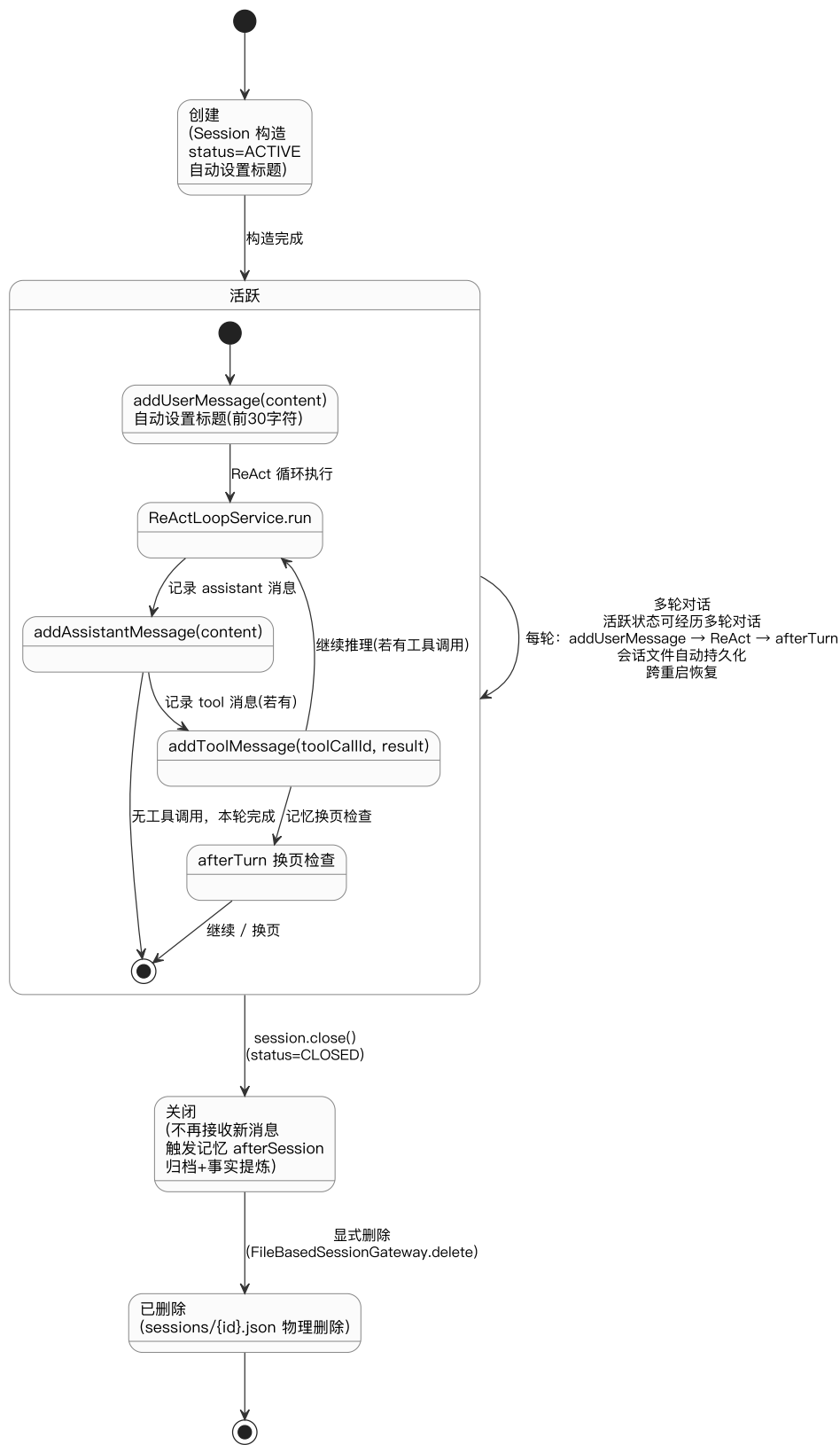
MCP 协议栈：

- `McpClientManager` 管理多个 MCP Server 连接（stdio / streamable_http 传输），启动期自动初始化。
 - 运行时管理： `McpClientManager` 支持 `disconnectServer`（关闭连接 + 注销其注册的工具）与 `reconnectServer`，Shell REPL 的 `/mcp` 命令（`list / connect / disconnect`）可直接查看与启停。
 - `McpToolRegistrar` 在启动期从 MCP Server 获取工具列表，注册为全局工具；工具调用时通过 JSON-RPC `callTool` 方法远程执行。
-

6.6 会话域

6.6.1 会话生命周期状态图

会话生命周期状态图



6.6.2 文字说明

会话域以 [Session](file:///Users/mawenbin/workspace/java/mwb_coding/mwb-ai-claw/mwb-ai-claw-domain/src/main/java/com/mwb/ai/claw/domain/core/Session.java) 聚合根为核心，管理对话上下文的生命周期：

状态流转：

- `CREATED` → `ACTIVE`：Session 构造时 `status=ACTIVE`，标题取第一条用户消息前 30 字符（若标题为空或以 `session-` 开头）。
- `ACTIVE`（多轮对话循环）：`addUserMessage` → `ReAct` 执行 → `addAssistantMessage`（+ `addToolMessage` 若有工具调用）→ `afterTurn`（记忆换页检查）。会话文件自动持久化，跨重启可恢复。
- `ACTIVE` → `CLOSED`：调用 `session.close()` 后 `status=CLOSED`，触发记忆域 `afterSession`（档案归档 + 事实提炼）。
- `CLOSED` → `DELETED`：通过 `FileBasedSessionGateway.delete()` 物理删除会话文件。

聚合行为约束：

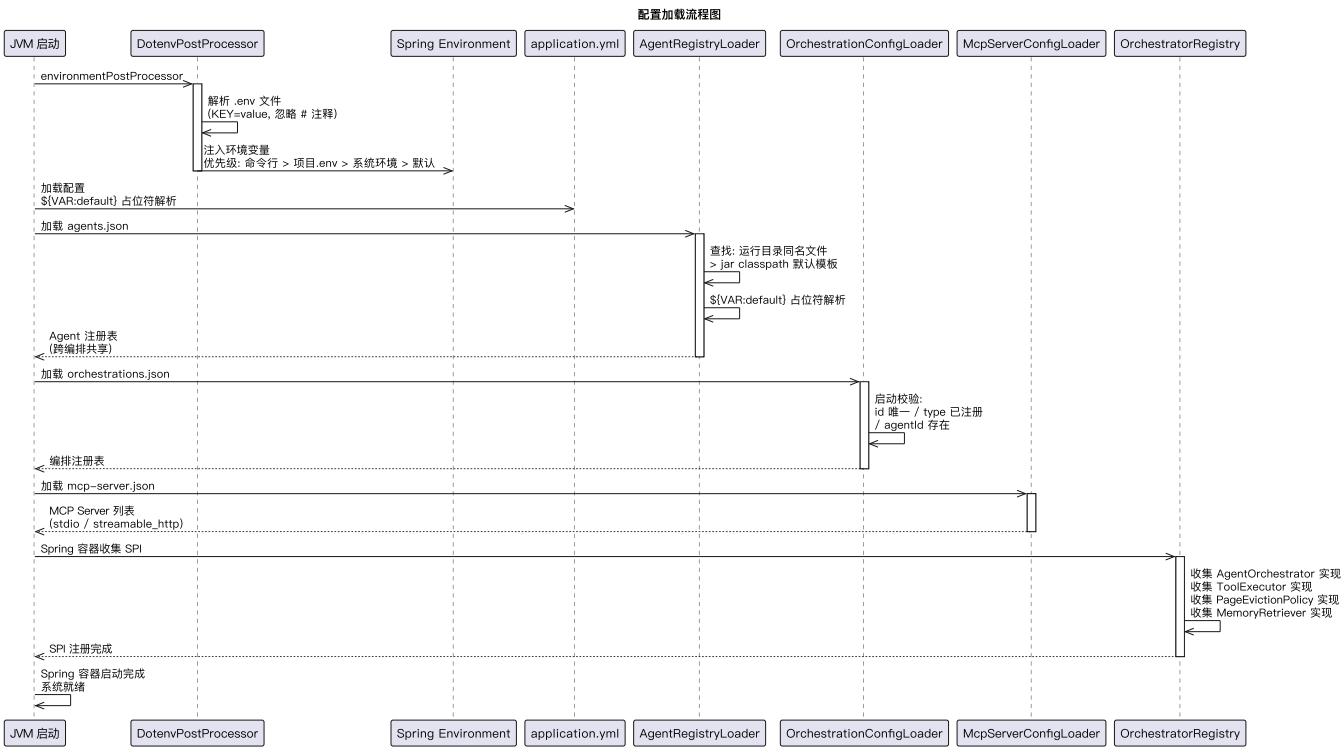
- 所有消息操作经 Session 的 `addUserMessage` / `addAssistantMessage` / `addToolMessage` / `close` 方法，每次操作刷新 `updateTime`。
- Session 持有 `messages` 列表，维护完整的对话历史（作为记忆系统的数据源）。

持久化：

- 会话 JSON 序列化到 `.agent/sessions/{sessionId}.json`。
- `FileBasedSessionGateway` 负责会话的增删改查，支持跨重启恢复。

6.7 配置加载域

6.7.1 配置加载流程图



6.7.2 文字说明

配置加载在 Spring Boot 启动期完成，分为环境变量注入、配置文件加载、注册表构建三个阶段：

环境变量注入：

- `DotenvEnvironmentPostProcessor` 在 Spring Environment 后处理阶段解析 `.env` 文件，注入为环境变量。
- 优先级（由高到低）：命令行参数 > 项目 `.env` 文件 > 系统环境变量 > 配置文件默认值。

配置文件加载：

- `application.yml`：Spring Boot 标准配置，`${VAR:default}` 占位符解析。

- `agents.json`：Agent 注册表，`AgentRegistryLoader` 加载。查找顺序：运行目录（`user.dir`）同名文件（命中即用，不再读取内置）> `jar classpath` 默认模板。支持 `${VAR:default}` 占位符。
- `orchestrations.json`：编排注册表，`OrchestrationConfigLoader` 加载。启动期校验：id 唯一、type 已注册、引用的 `agentId` 存在。
- `mcp-server.json`：MCP Server 配置，`McpServerConfigLoader` 加载。

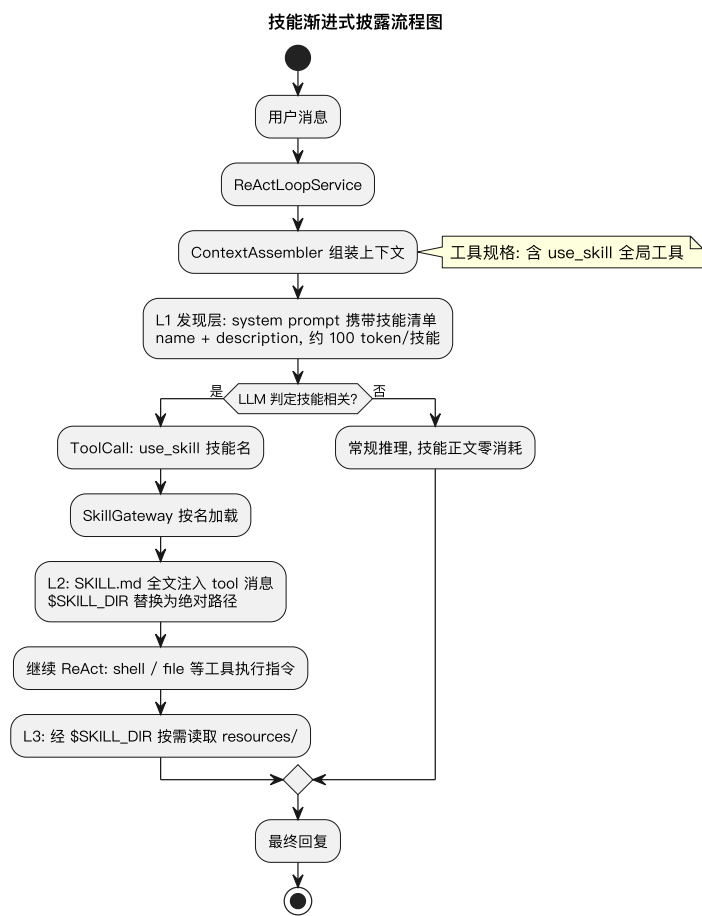
SPI 自动收集：

- Spring 容器启动完成后，`OrchestratorRegistry` 自动收集所有 `AgentOrchestrator` 实现并按 `type()` 注册。
- `DynamicToolRegistry` 自动收集所有 `ToolExecutor` 实现并注册为工具。
- `PageEvictionPolicy`、`MemoryRetriever` 同理自动收集。

配置覆盖优先级（由高到低）：命令行参数 > 项目 `.env` 文件 > 系统环境变量 > 配置文件默认值 > 代码默认值。

6.8 技能域 (Skill)

6.8.1 技能渐进式披露流程图



6.8.2 文字说明

技能域 (Skill) 遵循 **Agent Skills 开放标准**（Anthropic 于 2025-10 发布、2025-12 开放为 [AgentSkills.io](#) 标准）：将可复用的工作流 / 领域知识打包为 `skills/<name>/SKILL.md`（YAML frontmatter: `name / description` + Markdown 指令正文），采用渐进式披露（**Progressive Disclosure**）三层按需加载：

1. **L1 发现层**：启动时 `SkillLoader` 扫描技能根目录（运行目录 `skills/` > `classpath skills/` 兜底），解析 frontmatter 并启动校验（`name / description` 缺失、`name` 与目录不一致、`name` 重复 → 启动报错）。`SkillRegistryImpl` 维护 `name` → `Skill` 索引，`DefaultContextAssembler` 将技能清单（`name + description`，约 100 token/技能）注入 system prompt——LLM 据此判断「何时该用哪个技能」。

2. **L2 指令层**：LLM 判定任务匹配某技能描述时，调用 `use_skill(name)` 工具（全局工具，对齐 MCP，对所有 Agent 可见）按需加载 `SKILL.md` 全文，作为 tool 消息注入上下文继续 ReAct 推理。
3. **L3 资源层**：`SKILL.md` 正文中的 `$SKILL_DIR` 占位符由 `use_skill` 替换为技能目录绝对路径，脚本 / 参考文档 / 模板经既有 `file / shell` 工具按路径读取执行。

与现有能力的关系：工具（ToolExecutor / MCP）解决「能做什么」（always-on），技能解决「该怎么做」（on-demand）；技能只是指令注入，执行仍走工具沙箱（shell 白名单 / 路径限制 / 超时 / 截断），无特权提升；与编排（routing / conversational / delegate）正交——任意编排内 Agent 均可用技能，与记忆可叠加（技能可引导读写记忆）。

配置：`agent.skills-enabled`（总开关，默认 true，关闭后不加载技能、不注册 `use_skill`）、`agent.skills-dir`（技能根目录，默认 `${user.dir}/skills`）。内置 12 个技能位于 `start/src/main/resources/skills/` 作为 classpath 模板兜底：`code-review`（代码审查）、`project-structure-analysis`（项目结构分析）、`unit-test-writing`（单元测试编写）、`git-workflow`（Git 提交流程）、`ddd-modeling`（DDD 领域建模）、`tech-design-doc`（技术方案编写）、`web-research`（联网调研，配合 tavily MCP）、`database-design`（数据库设计）、`doc-writing-guide`（文档写作规范）、`markdown-diagramming`（mermaid 图表规范）、`doc-review`（文档审查与一致性）、`example-skill`（周报生成示例）。

新增技能（零代码）：在技能根目录放 `skills/<name>/SKILL.md`（frontmatter + 指令正文），重启应用即加载生效；技能目录内可含 `resources/` 子目录存放脚本 / 参考文档。